1. In React, you can handle file uploads by using an <input type="file" /> element.
   When a user selects a file, you can store that file in your component's state and send it to a server later (using FormData).

Example:

```
import { useState } from "react";

function FileUpload() {

const [file, setFile] = useState(null)

const handleChange = (event) => {

setFile(event.target.files[0]); // store selected file

 };

 const handleSubmit = (event) => {

 event.preventDefault();

 console.log("Uploaded file:", file);


   // If you were sending to a backend:

   // const formData = new FormData();

   // formData.append("myFile", file);

   // axios.post("/upload", formData);

 };


 return (

   <form onSubmit={handleSubmit}>

     <h3>File Upload Example</h3>
```

```
      <input type="file" onChange={handleChange} />
      <button type="submit">Upload</button>
    </form>
  );
}
```

2. Handling Dependent Dropdowns in React

A **dependent dropdown** means one dropdown's options change based on what's selected in another dropdown.
For example:

- Country → changes the list of States.

Example:

```
import { useState } from "react";

function DependentDropdown() {
  const countries = {
    India: ["Karnataka", "Maharashtra", "Kerala"],
    USA: ["California", "Texas", "Florida"],
  };

  const [selectedCountry, setSelectedCountry] = useState("");
  const [states, setStates] = useState([]);

  const handleCountryChange = (e) => {
    const country = e.target.value;
```

```
  setSelectedCountry(country);
  setStates(countries[country] || []);
};

return (
  <div>
    <h3>Dependent Dropdown Example</h3>

    <label>Country:</label>
    <select onChange={handleCountryChange}>
      <option value="">Select Country</option>
      {Object.keys(countries).map((country) => (
        <option key={country} value={country}>
          {country}
        </option>
      ))}
    </select>

    <br /><br />

    <label>State:</label>
    <select disabled={!selectedCountry}>
      <option value="">Select State</option>
```

```
        {states.map((state) => (
          <option key={state}>{state}</option>
        ))}
      </select>
    </div>
  );
}
```

---

3. Handling Multiple Checkboxes in React

If you have many checkboxes (like hobbies, skills, etc.), you can store them in an **array** in state.
When a checkbox is checked or unchecked, you update that array.

Example:

```
import { useState } from "react";

function MultipleCheckboxes() {
  const [selectedHobbies, setSelectedHobbies] = useState([]);

  const hobbies = ["Reading", "Traveling", "Gaming", "Cooking"];

  const handleChange = (event) => {
    const value = event.target.value;

    if (selectedHobbies.includes(value)) {
```

```
    // if already selected, remove it
    setSelectedHobbies(selectedHobbies.filter((hobby) => hobby !== value));
  } else {
    // if not selected, add it
    setSelectedHobbies([...selectedHobbies, value]);
  }
};

return (
  <div>
    <h3>Multiple Checkboxes Example</h3>
    {hobbies.map((hobby) => (
      <label key={hobby}>
        <input
          type="checkbox"
          value={hobby}
          checked={selectedHobbies.includes(hobby)}
          onChange={handleChange}
        />
        {hobby}
      </label>
    ))}
```

```
      <p>Selected Hobbies: {selectedHobbies.join(", ")}</p>
    </div>
  );
}
```